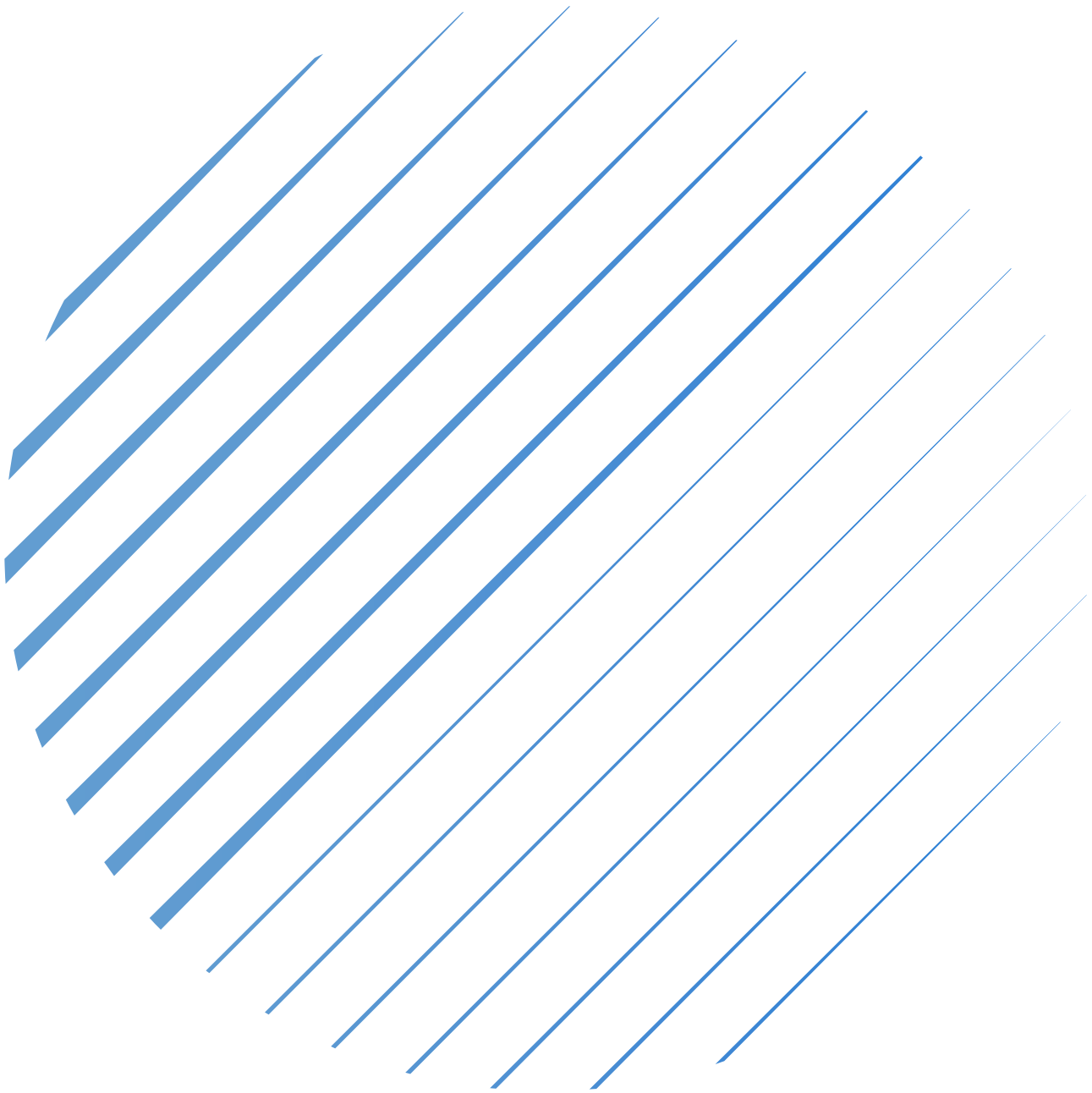
Ejercicio 1



15/09/2026
DAW2

Santiago González Vicente

INDICE

Ejercicio 1 1

1. Protocolos de comunicaciones: 3
2. Modelo de comunicaciones cliente – servidor y su relación con las aplicaciones web. 4
3. Estudio sobre los métodos de petición HTTP /HTTPS más utilizados. 10
4. Estudio sobre el concepto de URI (Identificador de Recursos Uniforme)/URL/URN, estructura, utilidad y relación con el protocolo HTTP/HTTPS. 14
5. Modelo de desarrollo de aplicaciones multicapa – comunicación entre capas – componentes – funcionalidad de cada capa. 16
6. Modelo de división funcional front-end / back-end para aplicaciones web. 18
7. Página web estática – página web dinámica – aplicación web – mashup 20
8. Componentes de una aplicación web. 23
9. Programas ejecutados en el lado del cliente y programas ejecutados en el lado del servidor - lenguajes de programación utilizados en cada caso. 24
10. Lenguajes de programación utilizados en el lado servidor de una aplicación web (características y grado de implantación actual). 25
11. Características y posibilidades de desarrollo de una plataforma XAMPP. 26
12. En que casos es necesaria la instalación de la máquina virtual Java (JVM) y el software JDK en el entorno de desarrollo y en el entrono de explotación. 27
13. IDE más utilizados (características y grado de implantación actual). 28
14. Servidores HTTP /HTTPS más utilizados (características y grado de implantación actual). 30
15. Apache HTTP vs Apache Tomcat 32
16. Navegadores HTTP /HTTPS más utilizados (características y grado de implantación actual) 34

1. Protocolos de comunicaciones:

-IP: El Protocolo de Internet (IP) es un protocolo, o conjunto de reglas, para enrutar y direccionar paquetes de datos para que puedan viajar a través de las redes y llegar al destino correcto. Los datos que atraviesan Internet se dividen en trozos más pequeños, llamados paquetes. La información IP se adjunta a cada paquete y esta información ayuda a los enrutadores a enviar los paquetes al lugar correcto. A cada dispositivo o dominio que se conecta a Internet se le asigna una dirección IP y a medida que los paquetes se dirigen a la dirección IP adjunta, los datos llegan a donde se necesitan.

Una vez que los paquetes llegan a su destino, se manejan de forma diferente en función del protocolo de transporte que se utilice en combinación con IP. Los protocolos de transporte más comunes son TCP y UDP.

-**TCP**: El protocolo de control de transmisión (TCP) es un protocolo de transporte, lo que significa que dicta la forma en que se envían y reciben los datos. Se incluye un encabezado TCP en la parte de datos de cada paquete que utiliza TCP/IP. Antes de transmitir los datos, TCP abre una conexión con el destinatario. TCP garantiza que todos los paquetes lleguen en orden una vez que se inicia la transmisión. Mediante TCP, el destinatario acusará recibo de cada paquete que llegue. Los paquetes que falten se enviarán de nuevo si no se acusa recibo.

TCP está diseñado para la fiabilidad, no para la velocidad. Como TCP tiene que asegurarse de que todos los paquetes lleguen en orden, la carga de datos mediante TCP/IP puede tardar más si faltan algunos paquetes.

TCP e IP se diseñaron originalmente para ser utilizados juntos, y a menudo se les conoce como el conjunto TCP/IP. Sin embargo, se pueden usar otros protocolos de transporte con IP.

-HTTP: El protocolo de transferencia de hipertexto (HTTP) constituye los cimientos de la red mundial, y se utiliza para cargar páginas web mediante enlaces de hipertexto. HTTP es un protocolo de capa de aplicación diseñado para transferir información entre los dispositivos conectados de la red, y se ejecuta sobre otras capas del conjunto de protocolos de la red. Un flujo típico sobre HTTP implica una máquina cliente que realiza una solicitud a un servidor, que a continuación envía un mensaje de respuesta.

-HTTPS: El protocolo de transferencia de hipertexto seguro (HTTPS) es la versión segura de HTTP, que es el principal protocolo utilizado para enviar datos entre un navegador web y un sitio web. El HTTPS está encriptado para aumentar la seguridad de las transferencias de datos. Esto es especialmente importante cuando los usuarios transmiten datos confidenciales, como al iniciar sesión en una cuenta bancaria, un servicio de correo electrónico o un proveedor de seguros médicos.

Cualquier sitio web, especialmente los que requieren credenciales de inicio de sesión, debe utilizar HTTPS. En los navegadores modernos, como Chrome, los sitios web que no utilizan HTTPS se señalan de forma diferente a los que sí lo hacen. Identifica un candado en la barra de URL que te indicará que la página web es segura. Los navegadores web se toman en serio el protocolo HTTPS.

2. Modelo de comunicaciones cliente – servidor y su relación con las aplicaciones web.

La arquitectura cliente-servidor es un modelo de diseño de software en el que las aplicaciones se dividen en dos componentes principales: clientes y servidores. Esta separación facilita la gestión de recursos y permite que ambos interactúen de manera eficiente. El cliente realiza solicitudes de servicios o datos, mientras que el servidor proporciona esos recursos. Este sistema se basa en una comunicación constante mediante protocolos de red, lo que permite un flujo de información bidireccional entre ambos.

Cliente

El cliente es el componente que inicia las solicitudes de servicio. Generalmente, se ejecuta en la máquina del usuario y puede tomar varias formas, como un navegador web, aplicaciones de escritorio o móviles. Su principal función es enviar peticiones al servidor y recibir las respuestas adecuadas.

Los clientes suelen presentar una interfaz de usuario que permite a las personas interactuar con los recursos y servicios proporcionados por el servidor. También pueden procesar ciertos datos localmente, pero dependen del servidor para acceder a información más compleja o voluminosa.

Servidor

El servidor es el componente que responde a las solicitudes que recibe del cliente. Este almacena y gestiona los recursos, que pueden incluir bases de datos, archivos y aplicaciones. La capacidad de procesamiento del servidor es crucial, ya que debe manejar múltiples peticiones simultáneamente y devolver respuestas de manera eficiente.

Los servidores pueden tener diferentes especificaciones y configuraciones, dependiendo de su propósito. Algunos se dedican exclusivamente a servir páginas web, mientras que otros pueden centrarse en la gestión de bases de datos o archivos.

Ventajas:

Centralización de datos

Una de las características más significativas de la arquitectura cliente-servidor es la centralización de la gestión de datos en el servidor. Esto garantiza que todos los clientes acceden a la misma información actualizada.

Reducción de conflictos de información

Al contar con una fuente única de datos, se minimiza la posibilidad de conflictos de información que pueden surgir en entornos donde los datos están dispersos. Los cambios realizados en el servidor se reflejan inmediatamente, evitando así versiones desactualizadas o inconsistentes de los datos entre diferentes clientes.

Integridad y consistencia de datos

La centralización contribuye a mantener la integridad y consistencia de los datos. Las operaciones, como la actualización y eliminación de registros, se llevan a cabo en un solo lugar, lo que reduce los riesgos de errores humanos y asegura que todos los clientes operan con la información más precisa.

Seguridad mejorada

La arquitectura cliente-servidor permite implementar medidas de seguridad más efectivas al concentrar la protección en el servidor. Esto es esencial para salvaguardar datos sensibles y garantizar el acceso seguro a la información.

Medidas de protección
Los servidores pueden estar protegidos por firewalls, sistemas de detección de intrusos y autenticación robusta. Estas capas de seguridad protegen los datos críticos ante accesos no autorizados, lo que es fundamental en la era digital actual.

Control de acceso
El control de acceso se puede gestionar de manera centralizada, lo que facilita la implementación de políticas de acceso a la información. Esto permite que solo los usuarios autorizados accedan a ciertos datos o recursos, aumentando así la seguridad general del sistema.

Escalabilidad
La arquitectura cliente-servidor es inherentemente escalable, lo que significa que tanto el cliente como el servidor pueden desarrollarse y ampliarse de manera independiente. Esta capacidad de adaptación es crucial para manejar el crecimiento de la demanda de servicios.

Desarrollo independiente
Dado que la arquitectura separa las funciones del cliente y el servidor, los desarrolladores pueden trabajar en cada uno sin afectar al otro. Esto permite implementar cambios y mejoras de manera más ágil, favoreciendo una rápida evolución de las aplicaciones.

Incremento de capacidad del servidor
En función de las necesidades, es posible aumentar la capacidad del servidor para manejar más conexiones y solicitudes sin que ello implique una modificación de los clientes. Esta flexibilidad permite que las empresas se adapten a la demanda de sus usuarios de forma eficiente.

Desventajas
Dependencia del servidor
La dependencia del servidor es una de las principales desventajas de esta arquitectura. Cuando los clientes están conectados a un servidor, su funcionalidad puede verse seriamente afectada si el servidor enfrenta algún problema.

Impacto de fallos del servidor
Si el servidor falla o experimenta una caída, todos los clientes que dependen de él quedan inoperativos. Este tipo de fallo puede interrumpir el flujo de trabajo en una organización, generando pérdidas tanto de tiempo como económicas. Los problemas de disponibilidad pueden dañar la confianza de los usuarios en el sistema.

Estrategias de mitigación
Para afrontar esta dependencia, es vital implementar estrategias de mitigación. Algunas de las prácticas más comunes incluyen:

-Utilización de servidores redundantes para asegurar la continua disponibilidad de servicios.
-Implementación de sistemas de respaldo y recuperación para minimizar el impacto de posibles fallos.
-Monitoreo constante del estado del servidor para anticipar y abordar problemas antes de que se conviertan en crisis.

Gestión de actualizaciones

La gestión de actualizaciones en la arquitectura cliente-servidor puede resultar un desafío considerable. Mantener el software de todos los clientes sincronizado con el servidor es esencial para garantizar la funcionalidad adecuada del sistema.

Desafíos en la actualización de clientes

Actualizar los clientes puede ser problemático, especialmente en entornos con un gran número de usuarios. Si no se realizan adecuadamente, las actualizaciones pueden generar incompatibilidades que afecten el rendimiento de las aplicaciones. Esto puede llevar a la frustración de los usuarios y a una disminución en la productividad.

Planificación de despliegues

Una planificación cuidadosa de los despliegues de software es crucial para garantizar que las actualizaciones se realicen de manera eficiente. Estrategias como:

-Realizar pruebas en entornos controlados antes de implementar en producción.
-Establecer horarios de mantenimiento que minimicen el impacto en los usuarios.
-Comunicar claramente a los usuarios los cambios y fechas de actualización.

Problemas de concurrencia

La gestión de múltiples peticiones simultáneas es otro desafío que enfrenta la arquitectura cliente-servidor. A medida que aumenta el número de clientes, el servidor debe ser capaz de manejar estas solicitudes de manera efectiva.

Manejo de múltiples peticiones

Si el servidor no está diseñado para gestionar un alto volumen de peticiones concurrentes, puede sufrir demoras, caídas o incluso fallos completos. Esto afecta negativamente la experiencia del usuario y puede provocar insatisfacción. Los sistemas deben ser lo suficientemente robustos para atender a todos los clientes simultáneamente.

Balanceo de carga

El balanceo de carga es una técnica que distribuye las solicitudes entre varios servidores, mejorando la capacidad de respuesta y la disponibilidad del sistema. Implementar un balanceador de carga puede mitigar muchos problemas asociados a las altas demandas de tráfico, garantizando que ningún servidor individual se convierta en un punto crítico de fallo.

Roles y responsabilidades en la arquitectura

La correcta asignación de roles y responsabilidades en la arquitectura cliente-servidor es fundamental para asegurar el buen funcionamiento del sistema. Cada componente desempeña funciones específicas que garantizan la eficiencia, la seguridad y la gestión efectiva de los recursos.

Función del cliente

El cliente es la interfaz que permite a los usuarios interactuar con las aplicaciones. Su función principal es enviar solicitudes al servidor y recibir la información adecuada para presentar los resultados de manera comprensible. Esta interacción se lleva a cabo a través de diferentes aplicaciones, como navegadores web o aplicaciones de escritorio.

-**Enviar solicitudes**: El cliente genera y envía peticiones al servidor cuando el usuario realiza alguna acción, como acceder a una página web o consultar datos.

-**Recibir y procesar respuestas**: Después de que el servidor procesa la solicitud, el cliente recibe la respuesta y la presenta al usuario de forma adecuada, lo que puede implicar la visualización de datos, imágenes o archivos.

-**Interfaz de usuario**: El cliente proporciona una interfaz gráfica que permite al usuario interactuar de manera intuitiva y eficiente con la aplicación, facilitando la navegación y la ejecución de tareas.

Función del servidor

El servidor actúa como el motor detrás de la arquitectura cliente-servidor. Su función es gestionar las solicitudes que recibe de los clientes y proporcionar los recursos y servicios necesarios para satisfacer dichas peticiones.

-**Procesar solicitudes**: El servidor escucha las solicitudes de los clientes y las procesa, lo que puede incluir búsquedas en bases de datos, cálculos o generación de contenido dinámico.

-**Almacenar recursos:** El servidor es responsable de administrar y almacenar todos los recursos necesarios para la aplicación, incluyendo bases de datos, archivos, y otros activos digitales.

-**Administrar la seguridad**: El servidor implementa medidas de seguridad para proteger los datos sensibles y controlar el acceso de los usuarios, asegurando que solo las personas autorizadas puedan acceder a información crítica.

Separación de responsabilidades

Un principio clave en la arquitectura cliente-servidor es la separación de responsabilidades entre el cliente y el servidor. Esta división permite un desarrollo modular y facilita el mantenimiento del sistema.

-**Desarrollo independiente**: El cliente y el servidor pueden desarrollarse de manera independiente, lo que permite a los equipos enfocar sus esfuerzos en mejorar cada componente sin tener que realizar cambios simultáneos en ambos.

-**Facilidad en la actualización**: La separación de responsabilidades facilita la implementación de actualizaciones. Los cambios en la lógica del servidor no requieren necesariamente modificaciones en el cliente y viceversa.

-**Optimización de recursos**: Esta separación también permite una mejor gestión de los recursos, ya que el servidor puede ser optimizado para manejar múltiples solicitudes sin que esto afecte el rendimiento del cliente.

Factores a considerar al diseñar una arquitectura cliente-servidor

Requerimientos de seguridad

La seguridad es un aspecto esencial en cualquier sistema que funcione bajo el modelo cliente-servidor. Los datos sensibles requieren protección para evitar accesos no autorizados.

Medidas de protección

Implementar mecanismos de seguridad robustos es crucial. Esto puede incluir el uso de cifrado para proteger los datos en tránsito y almacenamiento. Las conexiones seguras mediante protocolos como HTTPS y el uso de VPNs son prácticas comunes.

Control de acceso

El control de acceso garantiza que solo los usuarios autorizados puedan acceder a la información y a los recursos del servidor. Se pueden utilizar autenticaciones de múltiples factores y sistemas de gestión de identidades para mejorar esta protección.

Capacidad y escalabilidad

Un diseño adecuado debe contemplar la capacidad para manejar el crecimiento. La escalabilidad asegura que el sistema pueda soportar un aumento en el tráfico y en el número de usuarios sin degradar su rendimiento.

Desarrollo independiente

El cliente y el servidor pueden ser desarrollados y actualizados de manera independiente, lo que facilita la incorporación de nuevas funcionalidades y mejora continua del sistema sin afectar a su conjunto.

Incremento de capacidad del servidor

Los servidores deben ser capaces de ampliarse, ya sea a través de hardware adicional o mediante soluciones en la nube que permiten aumentar rápidamente los recursos disponibles según la demanda.

Gestión de datos

La forma en que se gestionan los datos en un entorno cliente-servidor es fundamental para garantizar la integridad y la disponibilidad. Es necesario establecer procedimientos claros para el manejo de la información.

Integridad y consistencia

Es vital que los datos sean precisos y se mantengan consistentes a lo largo del tiempo. Esto puede requerir la implementación de validaciones y controles de calidad en las operaciones de escritura y lectura.

Respaldo y recuperación

El establecimiento de procesos de respaldo regulares y un plan de recuperación ante desastres es esencial para garantizar que la información no se pierda y que el sistema pueda restaurarse rápidamente después de un fallo.

Soporte y mantenimiento

Finalmente, es crucial definir un plan de soporte y mantenimiento que garantice el correcto funcionamiento del sistema una vez implementado.

Mantenimiento proactivo

El mantenimiento debe incluir tareas preventivas para identificar y solucionar problemas antes de que afecten la operatividad del sistema. Esto puede abarcar actualizaciones de software y revisiones del rendimiento del servidor.

Capacitación y soporte técnico

Brindar capacitación adecuada al personal y contar con un soporte técnico competente son factores clave para resolver problemas de manera eficaz y garantizar que los usuarios aprovechen al máximo las herramientas disponibles.

3. Estudio sobre los métodos de petición HTTP /HTTPS más utilizados.

GET

El método GET solicita una representación de un recurso específico. Las peticiones que usan el método GET sólo deben recuperar datos.

Sintaxis
GET /index.html

Compatibilidad con navegadores

Report problems with this compatibility data • View data on GitHub

	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS
GET	✓ 1	✓ 12	✓ 1	✓ 2	✓ 1	✓ 18	✓ 4	✓ 10.1	✓ 1	✓ 1	✓ 1	✓ 1

Tip: you can click/tap on a cell for more information.

HEAD

El método HEAD pide una respuesta idéntica a la de una petición GET, pero sin el cuerpo de la respuesta.

Sintaxis

HEAD <request-target>["?"<query>] HTTP/1.1

Browser compatibility

Report problems with this compatibility data • View data on GitHub

	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS
HEAD	✓ 1	✓ 12	✓ 1	✓ 15	✓ 1	✓ 18	✓ 4	✓ 14	✓ 1	✓ 1	✓ 4.4	✓ 1

POST

El método POST se utiliza para enviar una entidad a un recurso en específico, causando a menudo un cambio en el estado o efectos secundarios en el servidor.

Sintaxis
POST <request-target>["?"<query>] HTTP/1.1

Browser compatibility

Report problems with this compatibility data • View data on GitHub

	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS
POST	✓ 1	✓ 12	✓ 1	✓ 15	✓ 1	✓ 18	✓ 4	✓ 14	✓ 1	✓ 1	✓ 4.4	✓ 1

PUT

El modo PUT reemplaza todas las representaciones actuales del recurso de destino con la carga útil de la petición.

Sintaxis
PUT <request-target>["?"<query>] HTTP/1.1

DELETE

El método DELETE borra un recurso en específico.

Sintaxis
DELETE <request-target>["?"<query>] HTTP/1.1

Browser compatibility

Report problems with this compatibility data • View data on GitHub

	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS
CONNECT	✓ 1	✓ 12	✓ 1	✓ 15	✓ 1	✓ 18	✓ 4	✓ 14	✓ 1	✓ 1	✓ 4.4	✓ 1

CONNECT

El método CONNECT establece un túnel hacia el servidor identificado por el recurso.

Sintaxis
CONNECT <host>:<port> HTTP/1.1

Browser compatibility

Report problems with this compatibility data • View data on GitHub

	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS
OPTIONS	✓ 1	✓ 12	✓ 1	✓ 15	✓ 1	✓ 18	✓ 4	✓ 14	✓ 1	✓ 1	✓ 4.4	✓ 1

OPTIONS

El método OPTIONS es utilizado para describir las opciones de comunicación para el recurso de destino.

Sintaxis
OPTIONS *|<request-target>["?"<query>] HTTP/1.1

TRACE

El método TRACE realiza una prueba de bucle de retorno de mensaje a lo largo de la ruta al recurso de destino.

Sintaxis
TRACE <request-target>["?"<query>] HTTP/1.1

PATCH

El método PATCH es utilizado para aplicar modificaciones parciales a un recurso.

Sintaxis
PATCH <request-target>["?"<query>] HTTP/1.1

4. Estudio sobre el concepto de URI (Identificador de Recursos Uniforme) /URL/URN, estructura, utilidad y relación con el protocolo HTTP/HTTPS.

URI

El URI (siglas de uniform resource identifier) o identificador uniforme de recursos (por su nombre en español) sirve para acceder a un recurso físico o abstracto por Internet.

Estructura

Un URI consta de un máximo de cinco partes, de las cuales solo dos son obligatorias:

-**scheme** (esquema): proporciona información sobre el protocolo utilizado.
-**authority** (autoridad): identifica el dominio.
-**path** (ruta): muestra la ruta exacta al recurso.
-**query** (consulta): representa la acción de consulta.
-**fragment** (fragmento): designa una parte del recurso principal.

Como ejemplo podemos poner una dirección web común: "https://example.org/test/test1?search=test-question#part2"
scheme: https
authority: example.org
path: test/test1
query: search=test-question
fragment: part2

Utilidad

Dependiendo de la situación, el recurso puede ser de muchos tipos: por ejemplo, un URI puede identificar tanto una página web como al remitente o al destinatario de un correo electrónico. Las aplicaciones utilizan este identificador único para interactuar con el recurso o consultar información sobre el mismo.

URL

La URL (siglas de Uniform Resource Locator, es decir, "Localizador Uniforme de Recursos") es una secuencia específica de caracteres que identifica y permite localizar y recuperar una información determinada en internet.

Estructura

-**Protocolo de red**. Es el segmento inicial de una URL, cuya labor es indicarle al sistema qué tipo de conexión debe establecer y cuál es el lenguaje específico que se hablará con el computador o la red de computadoras que brindarán la información solicitada. Http, Https, mailto y ftp son los principales protocolos web que encabezan una URL.

-**Subdominio**. Es una parte opcional del dominio que permite organizar o identificar secciones y servicios dentro de un sitio.

-**Dominio**. Es el "nombre" del servidor que se intenta contactar, es decir, el nombre de quien tiene lo que se busca.

-**Puerto**. Es la "puerta" técnica de entrada al servidor web, o sea, la especificación de por dónde se lleva a cabo el ingreso al computador de destino. Se suele expresar en términos como :80 o :443.

-**Ruta**. Es la dirección exacta del recurso buscado dentro del servidor, normalmente expresada como un conjunto de parámetros separados por barras (/), puntos (.) u otros signos especializados.

-**Parámetros adicionales**. En ocasiones una URL puede contener parámetros adicionales para especificar aún más el modo en que la información buscada debe ser recuperada. Normalmente se trata de claves (keys) y valores (values) separados con el signo ampersand (&), u otras referencias que indican colores o maneras de mostrar los datos.

-**Ancla**. Es un descriptor que, introducido con el símbolo numeral (#), especifica al sistema qué fragmento de la información presentada debe resaltarse o amerita un tratamiento especial.

Utilidad

Cada URL es una dirección específica dentro de internet, de modo que al conocerla se pueden localizar directamente distintas informaciones en línea. Así, al introducir la URL de una imagen se podrá visualizarla, mientras que la de un archivo específico permitirá descargarlo y la de un programa online ejecutarlo.

Por otro lado, los buscadores en internet manejan las URL de las páginas web (junto con otros datos de su localización) para identificar cuáles se ajustan a la información que se les ha solicitado, y es por ello que algunas páginas son más fáciles de encontrar que otras con este sistema.

URN

El urn (uniform resource name) es un formato que permite identificar claramente distintos tipos de objetos, algo especialmente útil en Internet, donde cada día interactúan millones de personas y aplicaciones –que también suelen intercambiar infinidad de recursos. Asignarles un código de identificación exclusivo evita confusiones y garantiza que todos los implicados se refieran al mismo objeto.

Estructura

-Todo URN comienza con la información sobre el esquema URN.
-Le sigue el namespace identifier (NID) o identificador del espacio de nombres, que debe figurar en el registro de la IANA y que puede subdividirse introduciendo otros dos puntos.
-Finalmente, se añade el namespace-specific string (NSS) o cadena específica del espacio de nombres, que identifica el objeto en cuestión.



5. Modelo de desarrollo de aplicaciones multicapa – comunicación entre capas – componentes – funcionalidad de cada capa.

La arquitectura multicapa o n-tier architecture es una estructura de diseño común en el desarrollo de aplicaciones, que se basa en dividir la lógica de la aplicación en capas independientes, con roles bien definidos. Esta segmentación permite organizar mejor el código, facilitar su mantenimiento y reutilización, y mejorar su escalabilidad y seguridad. Las aplicaciones multicapa se componen de diferentes niveles que colaboran para llevar a cabo operaciones complejas, y que se comunican de una manera específica y ordenada.

Cada capa cumple una función específica y se comunica con otras de una forma estructurada, asegurando que cada componente sea responsable de una sola tarea. Este modelo se utiliza ampliamente en aplicaciones Java desarrolladas con frameworks como Spring Boot, ya que favorece las buenas prácticas de desarrollo y el acoplamiento débil entre componentes, haciendo que las aplicaciones sean más fáciles de probar, mantener y escalar.

1. Controller

La capa Controller es la capa que actúa como intermediaria entre el usuario o cliente y la aplicación. Se encarga de recibir las solicitudes HTTP (usualmente peticiones REST) y de derivarlas hacia las capas adecuadas de la aplicación para su procesamiento. Un controlador no procesa la lógica de negocio ni accede directamente a la base de datos; su papel es delegar el trabajo a otras capas, especialmente a la capa de Service, que es donde reside la lógica de negocio. Al recibir una solicitud, el controlador suele:

- Extraer los datos necesarios de la solicitud.
- Llamar a los métodos correspondientes en la capa Service para manejar la lógica de negocio.
- Recibir la respuesta de la capa Service y generar una respuesta adecuada para el cliente.

2. Service

La capa Service es donde se encuentra la lógica de negocio de la aplicación, es decir, el conjunto de reglas y procedimientos que definen el funcionamiento interno del sistema. Aquí se procesan todas las operaciones necesarias para llevar a cabo las funciones requeridas por el cliente, como cálculos, validaciones, autorizaciones, entre otros. La capa Service se encarga de coordinar la interacción entre la capa Controller y la capa de acceso a datos (Repository).

3. Repository o DAO (Data Access Object)

La capa Repository o DAO es la capa de acceso a datos. Su objetivo es interactuar directamente con la base de datos y manejar la persistencia de los datos de la aplicación. En esta capa se encuentran los métodos necesarios para consultar, insertar, actualizar y eliminar datos en las tablas de la base de datos, abstractando la lógica de interacción con la base de datos.

4. Model o Entity

La capa Model o Entity contiene las clases que representan los objetos de negocio de la aplicación. Cada clase en esta capa define un tipo de entidad o dato de negocio, como un usuario, producto o pedido, y suele estar marcada con la anotación @Entity si está mapeada a una tabla en la base de datos. Cada instancia de estas clases puede ser guardada en la base de datos como un registro.

5. DTO (Data Transfer Object)

Los DTOs o Objetos de Transferencia de Datos son clases diseñadas específicamente para transferir datos entre las diferentes capas de la aplicación, y entre la aplicación y el cliente (frontend). A

diferencia de las entidades de la capa Model, los DTOs están destinados a ser estructuras de datos ligeras, sin lógica de negocio, y son útiles para evitar exponer directamente las entidades del negocio al cliente o frontend.

6. Modelo de división funcional front-end / back-end para aplicaciones web.

- **front-end**: El término front end hace referencia a la interfaz gráfica de usuario (GUI) con la que los usuarios pueden interactuar de forma directa, como los menús de navegación, los elementos de diseño, los botones, las imágenes y los gráficos. En términos técnicos, una página o pantalla que el usuario ve con varios componentes de la interfaz de usuario se denomina modelo de objetos del documento (DOM).
- **back-end**: A veces denominado servidor, el back end de la aplicación administra la funcionalidad general de la aplicación web. Cuando el usuario interactúa con el front end, la interacción envía una solicitud al back end en formato HTTP. El backend procesa la solicitud y devuelve una respuesta.
- **Diferecias Claves**:

	Front end	Back end
Telefonías	El front end utiliza HTML, CSS, JavaScript y marcos de front end.	El back end utiliza lenguajes de programación como Java, Python, Ruby, API y sistemas de administración de bases de datos.
Simultaniedad	Cada usuario tiene su propia copia de una aplicación, por lo que el front end no tiene que gestionar los problemas de concurrencia.	El back end utiliza varias estrategias para gestionar miles de solicitudes de usuarios al mismo tiempo.
Almacenamiento en caché	Los navegadores o las aplicaciones cliente almacenan en caché los archivos de la aplicación y los utilizan para mejorar el rendimiento.	Los sistemas de back end almacenan en caché los archivos en diferentes servidores o en una CDN.
Seguridad	La seguridad en el desarrollo del front end es principalmente responsabilidad del usuario. Los desarrolladores de front end se centran principalmente en los flujos de trabajo de validación de entradas de usuario y autenticación de usuarios.	La seguridad en el desarrollo del back end es más completa para proteger las bases de datos, los servicios de back end y la propia aplicación. Se logra mediante el cifrado, los sistemas de autenticación seguros y las prácticas de codificación seguras.
Objetivos de desarrollo	El desarrollo del front end se centra en crear interfaces de usuario totalmente funcionales, con buena capacidad de respuesta y bien diseñadas.	El desarrollo del back end implica la creación de una arquitectura fiable que respalde el desarrollo del front end.
Habilidades de desarrollo	Los desarrolladores de front end conocen HTML, CSS y JavaScript. Pueden utilizar marcos de front end y crear páginas visualmente atractivas. Estos marcos desentrañan los puntos débiles de los usuarios a medida que interactúan con la aplicación.	Los desarrolladores de back end cuentan con habilidades de codificación y administración de bases de datos. También entienden la seguridad del código y cómo utilizar las herramientas, plataformas y marcos de desarrollo de aplicaciones.

7. Página web estática – página web dinámica – aplicación web – mashup.

- **Página web estática**: hacen referencia a aquellas aplicaciones web en las que el usuario recibe una página web cuya interacción no conlleva ningún tipo de acción, ni en la propia página, ni genera respuesta alguna por parte del servidor. Este tipo de aplicaciones suele realizarse utilizando el lenguaje HTML exclusivamente para la organización visual de la información.

- Ventajas:

Velocidad y rendimiento	Los sitios estáticos son rápidos y tienen mejor rendimiento. Esto se debe a que entregan contenido sin necesidad de pasos adicionales como procesamiento del servidor o búsquedas en la base de datos.
Beneficios de seguridad	Sitios web estáticos son más seguros. Esto se debe al hecho de que no usan bases de datos o lenguajes de scripting del lado del servidor (como PHP).
Facilidad de alojamiento	Los sitios estáticos tienen una estructura simple y consisten en archivos HTML y CSS code for design. Algunos también pueden tener archivos Javascript para funciones dinámicas básicas. Básicamente, subes tu página a un servidor, y listo. Sin configuraciones complicadas o ajustes constantes. Sencillo y fácil de administrar, para que puedas centrarte en lo que realmente importa.

- Desventajas:

Funcionalidad Limitada	Los sitios web estáticos pueden ser un poco limitantes si estás buscando características sofisticadas. Son excelentes para el diseño básico, pero no tan útiles si necesitas actualizaciones en vivo.
Retos de la gestión de contenido	Cada vez que algo cambia – tienes que editar manualmente el sitio. No es imposible, pero definitivamente es más trabajo que usar un sistema de gestión de contenido (CMS) como WordPress, que te permite actualizar las cosas sobre la marcha.
Problemas de escalabilidad	Los sitios estáticos también pueden chocar con una pared cuando se trata de crecimiento. Son increíbles para necesidades sencillas, pero si tu sitio comienza a volverse complejo, es posible que debas cambiar el enfoque por completo. Puede que te resulte complicado ampliar sin una revisión completa.
Falta de personalización	Además, cuando se trata de crear una experiencia personalizada, los sitios estáticos se quedan cortos. Son como una talla única, todos obtienen lo mismo. Si necesitas que tu sitio se adapte a diferentes usuarios u ofrecer una experiencia

	personalizada, los sitios estáticos no son la mejor opción.
Gastos generales de mantenimiento	Por último, mantener un sitio estático puede ser una molestia a medida que crece. Imagina el pavor si necesitas actualizar cien páginas individuales a mano. Puede ser tedioso y lento, especialmente si estás trabajando con mucho contenido que cambia a menudo.

- **Página web dinámica**: la programación de estas aplicaciones suele conocerse con el nombre de HTML dinámico (DHTML) y se refiere a aquellas aplicaciones en las que la interacción del cliente con el recurso recibido por parte del servidor (página web) produce algún tipo de cambio en la visualización del mismo (cambios de formato, ocultación de partes del documento, creación de elementos nuevos, etc.). Los lenguajes involucrados en este tipo de aplicaciones incluyen, entre otros, HTML, CSS o las múltiples variaciones de JavaScript (VBScript, JScript, Flash, etc.).

- Ventajas:

Personalización de contenido	Los sitios web dinámicos muestran a los usuarios contenido diferente basado en sus preferencias o comportamiento.
Funciones interactivas	Formularios, comentarios y carritos de compra son sólo algunos ejemplos de elementos interactivos que puedes encontrar en páginas web dinámicas como sitios de comercio electrónico.
Actualizaciones fáciles	Con un sitio dinámico basado en un sistema de gestión de contenido (CMS), puedes actualizar fácilmente contenido sin necesidad de cambiar el código subyacente.
Mejor para SEO	Los sitios web dinámicos pueden generar automáticamente URLs amigables para la búsqueda y meta tags. Como resultado, esto mejora la visibilidad del sitio en los motores de búsqueda y atrae más tráfico orgánico. Además, las páginas web dinámicas soportan un diseño responsive para adaptarse a varios tamaños de pantalla.
Escalabilidad	Los sitios web dinámicos pueden crecer con tus necesidades. Te permiten implementar nuevas características y añadir nuevas páginas sin necesidad de una rediseñar.
Integración de datos	Por ahora, puedes ver lo dinámicos que son los sitios web para dar a los usuarios una experiencia personalizada. La integración de datos juega un papel importante. Es como cuando todas las diferentes partes de tu sitio, como el inventario, los perfiles de usuario y el seguimiento de pedidos, se comunican entre sí.

- Desventajas:

Costos de desarrollo más altos	Construir un sitio web dinámico a menudo requiere más tiempo y experiencia, lo que lo hace más caro que los sitios estáticos.
Tiempos de carga más lentos	Debido a que las páginas dinámicas extraen datos de un servidor, pueden tardar más en cargar, especialmente si el servidor está ocupado o la conexión a Internet es lenta. Además, las grandes bases de datos y el diseño sofisticado pueden hacer que el sitio web sea pesado, lo que requiere más tiempo para cargar para tus visitantes.
Mantenimiento complejo	Mantener un sitio dinámico funcionando sin problemas requiere actualizaciones y mantenimiento regulares.
Riesgos de seguridad superiores	Con más partes móviles, los sitios web dinámicos pueden ser especialmente vulnerables a amenazas de seguridad como la piratería.
Mayor uso de recursos	Debido a su intrincada configuración, los sitios web dinámicos a menudo necesitan más potencia del servidor y ancho de banda. Esto significa que tus costos de alojamiento pueden ser más altos en comparación con un sitio web estático. Es como tener un coche con un motor más grande: puede hacer más, pero también necesita más gasolina.

- **Aplicación web**: al contrario que en los tipos de aplicaciones anteriores, donde la interacción del usuario produce un cambio en el recurso recibido, las aplicaciones web interactivas se basan en que dicha interacción hace que se genere un diálogo entre el cliente y el servidor

- **Mashup**: es una página web o aplicación web que utiliza contenido de más de una fuente para crear un nuevo servicio único que se muestra en una sola interfaz gráfica.

8. Componentes de una aplicación web.

Front-end (lado del cliente): la parte visual con la que interactúan los usuarios, creada con HTML, CSS y JavaScript.

Backend (lado del servidor): se encarga de la lógica, la autenticación, las consultas a la base de datos y la comunicación con el servidor.

Base de datos: almacena y recupera datos de los usuarios, como perfiles, contenido o configuraciones.

API: conectan la aplicación web con herramientas externas o servicios internos para obtener datos y funciones en tiempo real.

Servidores web: reciben y procesan las solicitudes del cliente y envían las respuestas.

Equilibrador de carga: distribuye el tráfico entrante para garantizar el rendimiento y evitar la sobrecarga.

Red de entrega de contenidos (CDN): entrega archivos estáticos como imágenes o scripts más rápidamente utilizando servidores globales.

Caché: acelera la entrega de contenido almacenando temporalmente los datos a los que se accede con frecuencia.

9. Programas ejecutados en el lado del cliente y programas ejecutados en el lado del servidor - lenguajes de programación utilizados en cada caso.

Programas en el Lado del Cliente (Client-Side)
Estos programas se encargan de la interfaz visual, el diseño, la experiencia de usuario y las validaciones inmediatas en la pantalla. El código es descargado y ejecutado directamente por el navegador.
-Lenguajes y tecnologías principales:
· HTML: Lenguaje de marcado para estructurar el contenido de la página web.
· CSS: Lenguaje de diseño para dar estilo, colores y adaptabilidad visual.
· JavaScript: El lenguaje principal de comportamiento e interactividad.
· TypeScript: Una versión avanzada y más estricta de JavaScript.
· Frameworks de JavaScript: Herramientas como React, Angular o Vue.js, que también corren en el navegador.
Programas en el Lado del Servidor (Server-Side)
Estos programas procesan las peticiones de los usuarios, se conectan con las bases de datos, realizan operaciones de seguridad y generan el contenido dinámico que luego se envía al cliente.
-Lenguajes principales:
· JavaScript (Node.js): Permite usar JavaScript también para crear la lógica del servidor.
· PHP: Muy popular para la gestión de contenidos y bases de datos.
· Python: Utilizado con frameworks como Django o Flask por su potencia y facilidad.
· Java: Frecuente en grandes aplicaciones empresariales y bancarias.
· C# (.NET): Usado para desarrollar aplicaciones web robustas en entornos de Microsoft.
· Ruby: Conocido por su uso en el framework Ruby on Rails.

10. Lenguajes de programación utilizados en el lado servidor de una aplicación web (características y grado de implantación actual).

CGI	Es el abuelo de todos los sistemas para la programación de páginas de forma dinámica de servidor. Se escriben en lenguaje Perl pero, también pueden ser empleados otros lenguajes como Visual basic, C o C++.
Perl	Es un lenguaje de programación interpretado, lo que viene a ser que el código de los scripts en Perl no se compila, sino que cada vez que se va a ejecutar se lee el código y arranca interpretando lo que hay escrito. Es muy dinámico, ya que desde Perl podemos llamar a otros subprogramas escritos en otros lenguajes.
ASP	Es un lenguaje desarrollado por Microsoft para la creación de páginas dinámicas del servidor. Escribe en la propia web utilizando el lenguaje Visual Basic Script o Jscript.
PHP	Es un lenguaje gratuito e independiente de plataforma, rápido, con una librería de funciones enorme y con mucha documentación.
JSP	Es una tecnología orientada a crear páginas web con programación en Java, lenguaje esencialmente multiplataforma. Podemos crear aplicaciones web que se ejecuten en variados servidores web.

11. Características y posibilidades de desarrollo de una plataforma XAMPP.

XAMPP es un paquete de software que incluye Apache, MariaDB, PHP y Perl, lo que le permite ofrecer todos los componentes necesarios para desarrollar aplicaciones web con total garantía.

Caracteristicas	
Facilidad de uso y configuración	Descargar XAMPP e instalar el software es muy sencillo. Solo debes descargar el paquete desde su web oficial, ejecutar el instalador y seguir unos pasos que veremos más adelante. No necesitas conocimientos avanzados para configurar y empezar a utilizar XAMPP, lo que hace accesible para todos los niveles de experiencia.
Portabilidad y compatibilidad multiplataforma	XAMPP es compatible con diferentes sistemas operativos, incluyendo Windows, macOS y Linux, ¡podrás instalar XAMPP en Ubuntu sin ningún tipo de problema! Esto permite trabajar en diferentes entornos sin tener que preocuparse por problemas de compatibilidad. Además, la portabilidad de XAMPP facilita el traslado de proyectos entre diferentes ordenadores y sistemas operativos.
Soporte para múltiples lenguajes de programación y bases de datos	XAMPP incluye soporte para lenguajes de programación como PHP y Perl, además de gestionar bases de datos a través de MariaDB y MySQL. Esto permite a cualquier desarrollador trabajar con las tecnologías que prefiera, lo que a su vez proporciona un entorno fiebre y versátil para la creación de aplicaciones.

12. En qué casos es necesaria la instalación de la máquina virtual Java (JVM) y el software JDK en el entorno de desarrollo y en el entorno de explotación.

Entorno de Desarrollo:
· El Java Development Kit (JDK) contiene las herramientas de desarrollo esenciales, como el compilador (javac) que transforma el código fuente en bytecode (.class), además de librerías y depuradores.

· El JDK incluye internamente la Máquina Virtual de Java (JVM) (dentro del JRE), la cual es necesaria para probar y ejecutar el programa en la máquina del programador mientras se está programando.

Entorno de Explotación
· JVM es la encargada exclusiva de leer, interpretar y ejecutar el bytecode compilado en el sistema operativo del servidor o cliente. Sin la JVM, el programa en Java no puede funcionar.

· En producción no se compila código nuevo ni se necesitan herramientas de desarrollo. Instalar un JDK completo en producción es innecesario y desaconsejable por motivos de seguridad y optimización de recursos, ya que solo se precisa el entorno de ejecución (JRE o la JVM).

· Excepción para el JDK en explotación: Solo se requeriría instalar un JDK en producción si la aplicación en ejecución necesita compilar código fuente sobre la marcha (por ejemplo, motores de reglas dinámicos, servidores de aplicaciones con compilación integrada de JSP avanzados o herramientas de diagnóstico profundo que lo exijan).

13. IDE más utilizados (características y grado de implantación actual).

Nombre	Descripcion	Ventajas	Desventajas
Eclipse	Eclipse es uno de los IDE más reconocidos y utilizados en la comunidad Java. Es gratuito, de código abierto y compatible con múltiples plataformas, incluyendo Windows, macOS y Linux. Su arquitectura basada en plugins permite una extensibilidad significativa, adaptándose a las necesidades específicas de cada desarrollador.	Comunidad activa con amplia documentación. Gran cantidad de plugins disponibles. Soporte para múltiples lenguajes de programación.	Puede consumir muchos recursos del sistema. Curva de aprendizaje pronunciada debido a su complejidad.
IntelliJ IDEA	Desarrollado por JetBrains, IntelliJ IDEA es reconocido por su inteligencia y eficiencia. Ofrece dos versiones: una gratuita (Community Edition) y una de pago (Ultimate Edition) con funcionalidades adicionales. Es compatible con Windows, macOS y Linux.	Autocompletado avanzado y asistencia inteligente para código. Integración con herramientas de desarrollo como Maven y Gradle. Interfaz intuitiva y moderna.	Alto consumo de memoria y CPU. La versión Ultimate es de pago.
Apache NetBeans	Apache NetBeans es un IDE gratuito y de código abierto que ofrece soporte para múltiples lenguajes de programación, incluyendo Java, PHP y HTML5. Es compatible con Windows, macOS y Linux.	Fácil de instalar y configurar. Entorno amigable para principiantes. Soporte para múltiples lenguajes.	Puede ser más lento en comparación con otros IDE. No tiene tantas opciones de personalización como Eclipse o IntelliJ IDEA.
BlueJ	BlueJ es un IDE diseñado especialmente para principiantes en la programación con Java. Es ampliamente utilizado en entornos educativos debido a su simplicidad	Interfaz sencilla y fácil de entender. Ideal para aprender los conceptos básicos de Java.	Funcionalidades limitadas para proyectos avanzados. No es adecuado para desarrollos grandes o empresariales.

	y diseño orientado a objetos.	Permite visualizar la estructura de los objetos y clases.	
JDeveloper	JDeveloper es un IDE desarrollado por Oracle, diseñado principalmente para la creación de aplicaciones empresariales basadas en Java y otras tecnologías de Oracle.	Integración total con herramientas y productos de Oracle. Buen soporte para el desarrollo de aplicaciones empresariales. Incluye herramientas avanzadas de desarrollo y depuración.	Puede ser complejo para quienes no están familiarizados con el ecosistema de Oracle. Consume bastantes recursos del sistema.

14. Servidores HTTP /HTTPS más utilizados (características y grado de implantación actual).

Nombre	Descripcion	Caracteristicas
Nginx	Nginx es un servidor web de código abierto y de alto rendimiento, lanzado en 2004. Se ha convertido en uno de los servidores web más utilizados, junto con Apache. Nginx es una solución ideal para gestionar sitios web de alto tráfico. Muchos sitios y aplicaciones web de alta visibilidad, como Netflix o Pinterest, utilizan el servidor web Nginx. A fecha de mayo de 2022, Nginx posee un 33,5 % del mercado según W3Techs y un 30,71 % según Netcraft.	Gratuito y de código abierto Arquitectura asíncrona basada en eventos Alto rendimiento Bajo consumo de memoria Compatibilidad con IPv6
Apache HTTP Server	El servidor web Apache, oficialmente conocido como Apache HTTP Server, se lanzó en 1995. Apache es un servidor web gratuito y de código abierto; desarrollado y mantenido por la Apache Software Foundation (ASF). Es uno de los servidores web más populares del mundo. A fecha de mayo de 2022, Apache posee un 31,5 % del mercado según W3Techs y un 22,99 % según Netcraft.	Gratuito y de código abierto Arquitectura basada en módulos Fácil configuración y personalización Actualizaciones y parches de seguridad regulares Gran comunidad de desarrolladores Compatibilidad con IPv6
LiteSpeed	LiteSpeed es un servidor web propietario, de pago desarrollado por LiteSpeed Technologies; lanzado en 2003. Dispone de tres licencias: Free Starter, Site Owner y Web Host. Es una alternativa de pago para sitios web de alto tráfico, que ofrece un alto rendimiento y gran escalabilidad. A fecha de mayo de 2022, LiteSpeed posee un 12,1 % del mercado según W3Techs y un 4,35 % según Netcraft.	Arquitectura basada en eventos Ligero Configuración sencilla Características anti-DDoS integradas Mantenimiento sin interrupciones
Microsoft-IIS	Microsoft Internet Information Services es un servidor web de Microsoft, lanzado en 1995. Se desarrolló para usarlo con Windows NT. Su arquitectura	Arquitectura basada en módulos Seguridad y control Gestión remota

	modular permite añadir y quitar extensiones/módulos con facilidad. Además cuenta con múltiples características de seguridad y mecanismos de autenticación integrados. Microsoft-IIS se usa a menudo para alojar sitios estáticos y aplicaciones web ASP.NET. A fecha de mayo de 2022, Microsoft-IIS posee un 6 % del mercado según W3Techs y un 3 % según Netcraft.	
OpenResty	OpenResty es un servidor web, con una arquitectura basada en eventos, basado en Nginx y LuaJIT. Está diseñado para construir aplicaciones y servicios web escalables y de alto rendimiento, así que es capaz de soportar miles de conexiones al mismo tiempo. A fecha de mayo de 2022, Microsoft-IIS posee un 8,03 % del mercado según Netcraft.	Arquitectura basada en eventos Alto rendimiento

15. Apache HTTP vs Apache Tomcat

Diferencias		
Implementación	Apache HTTP Server es un servidor web de propósito general que puede servir archivos estáticos y admite varios lenguajes de scripting como PHP, Perl y Python. Puede utilizarse para alojar cualquier tipo de sitio web.	Apache Tomcat es un servidor de aplicaciones web diseñado específicamente para ejecutar servlets Java y páginas JavaServer Pages (JSP). Se utiliza principalmente para alojar aplicaciones web Java.
Compatibilidad con protocolos	Apache HTTP Server admite una amplia gama de protocolos, incluidos HTTP, HTTPS, FTP y SMTP. Ofrece mayor versatilidad en cuanto a la compatibilidad con protocolos.	Apache Tomcat admite principalmente los protocolos HTTP y HTTPS para servir aplicaciones web. Proporciona un contenedor de servlets y un motor JSP para gestionar las solicitudes, procesar los servlets y renderizar contenido dinámico.
Módulos y extensiones	Apache HTTP Server cuenta con una arquitectura modular que permite a los usuarios añadir o eliminar módulos según sus necesidades. Ofrece una amplia biblioteca de módulos para mejorar su funcionalidad, como mod_rewrite para la reescritura de URL, mod_ssl para el cifrado SSL/TLS y mod_proxy para el proxy inverso.	Apache Tomcat se centra en proporcionar las características esenciales para las aplicaciones web Java y dispone de un conjunto de extensiones más limitado.
Rendimiento	Apache HTTP Server es conocido por su alto rendimiento y escalabilidad. Está optimizado para servir contenido estático de manera eficiente y puede manejar un gran número de conexiones concurrentes.	Apache Tomcat, al ser un servidor basado en Java, requiere recursos y sobrecarga adicionales para ejecutar servlets y JSPs de Java. Por lo tanto, puede que no sea tan eficiente al servir archivos estáticos como Apache HTTP Server.
Configuración	Apache HTTP Server cuenta con un sistema de configuración altamente flexible y personalizable. Utiliza un archivo de configuración llamado httpd.conf, que permite a los usuarios definir diversas configuraciones del servidor, hosts virtuales y reglas de reescritura de URL.	Apache Tomcat tiene un mecanismo de configuración similar, pero enfocado en la configuración de aplicaciones web Java, como la definición de contextos, asignaciones de servlets y configuraciones de recursos.
Casos de uso	Apache HTTP Server se utiliza comúnmente para alojar sitios web estáticos, sitios web dinámicos con lenguajes de	Apache Tomcat está diseñado específicamente para alojar aplicaciones web basadas en Java. Proporciona un entorno de

	scripting y como servidor proxy inverso. También puede utilizarse como balanceador de carga junto con otros servidores.	ejecución para ejecutar servlets y JSPs de Java y se utiliza ampliamente en aplicaciones empresariales.

16. Navegadores HTTP /HTTPS más utilizados (características y grado de implantación actual).

Navegador	Descripcion	Caracteristicas
Google Chrome	Google Chrome se ha consolidado como uno de los navegadores más utilizados a nivel mundial debido a su combinación de velocidad, seguridad y extensibilidad. Su interfaz intuitiva y su capacidad para integrarse con otros servicios de Google lo hacen una opción atractiva tanto para usuarios casuales como para profesionales.	Velocidad: Chrome es conocido por ser uno de los navegadores más rápidos del mercado, optimizando el tiempo de carga de las páginas y el rendimiento general del navegador. Interfaz de usuario amigable: su diseño minimalista y limpio facilita la navegación, permitiendo a los usuarios encontrar con rapidez lo que necesitan sin distracciones innecesarias. Desarrollo continuo: Google invierte de forma constante en mejorar Chrome, añadiendo nuevas características y mejorando la seguridad y el rendimiento con cada actualización.
Safari	Safari es el navegador web desarrollado por Apple y optimizado para sus dispositivos, incluyendo iPhone, iPad y Mac. Este navegador destaca por su eficiencia energética, velocidad y profunda integración con el ecosistema de Apple, proporcionando una experiencia de navegación fluida y consistente.	Eficiencia energética: safari está optimizado para consumir menos energía, lo que prolonga la duración de la batería en dispositivos portátiles como MacBook y iPads. Esto lo hace ideal para usuarios que necesitan navegar durante largos periodos sin preocuparse por la carga. Privacidad y seguridad: Apple se centra en la privacidad del usuario. Safari incluye características como el bloqueo de rastreadores de terceros, protección contra fingerprinting, y un Informe de Privacidad que muestra los rastreadores bloqueados. Velocidad y rendimiento: gracias a su motor de JavaScript, Safari ofrece una

		navegación rápida y eficiente. Los tiempos de carga de las páginas son reducidos y el rendimiento general del navegador es excepcional.
Mozilla Firefox	Mozilla Firefox es un navegador web conocido por su fuerte enfoque en la privacidad del usuario y la personalización. Desarrollado por Mozilla, una organización sin fines de lucro, Firefox se destaca por ofrecer una experiencia de navegación segura y flexible.	Privacidad y seguridad: Firefox ofrece una robusta protección contra el rastreo en línea, incluyendo el bloqueo de cookies de terceros, rastreadores de anuncios y huellas digitales. Su navegador integrado permite una navegación aún más privada. Personalización: los usuarios pueden personalizar Firefox con una vasta colección de extensiones y temas disponibles en su tienda. Esto incluye herramientas para productividad, seguridad y personalización visual. Rendimiento y velocidad: con un motor de renderizado avanzado, Firefox proporciona tiempos de carga rápidos y una experiencia de navegación fluida, incluso con múltiples pestañas abiertas.
Microsoft Edge	Microsoft Edge es el navegador web desarrollado por Microsoft, diseñado para reemplazar a Internet Explorer y competir con los navegadores modernos en términos de velocidad, seguridad y características avanzadas. Con su base en Chromium, Edge combina la compatibilidad con un rendimiento mejorado y una integración perfecta con el ecosistema de Microsoft.	Velocidad y rendimiento: Edge está construido sobre el motor Chromium, lo que le permite ofrecer tiempos de carga rápidos y un rendimiento fluido. Su optimización para Windows asegura que aproveche al máximo el hardware del sistema. Seguridad y privacidad: Microsoft Edge incluye características avanzadas de seguridad como SmartScreen, que protege contra sitios web maliciosos y phishing. Además, ofrece herramientas de privacidad como el bloqueo de rastreadores y la prevención de huellas digitales.

		Integración con Microsoft 365: Edge se integra a la perfección con los servicios de Microsoft 365, facilitando el acceso y la gestión de documentos en la nube, así como otras herramientas de productividad.
Brave	Brave es un navegador web que pone un fuerte énfasis en la privacidad y la seguridad del usuario, ofreciendo una experiencia de navegación rápida y libre de anuncios invasivos. Desarrollado con un enfoque en la protección de datos personales, Brave se destaca por su capacidad para bloquear rastreadores y mejorar el rendimiento de la navegación.	Privacidad y seguridad: Brave bloquea automáticamente rastreadores y anuncios de terceros, protegiendo al usuario de la recopilación no deseada de datos. También ofrece una VPN integrada para encriptar las conexiones y una cartera digital para gestionar criptomonedas de manera segura. Velocidad y rendimiento: sin la carga de anuncios y rastreadores, las páginas web se cargan entre 3 y 6 veces más rápido que en otros navegadores. Esto no solo mejora la velocidad de navegación, sino que también ahorra datos y batería. Recompensas y criptomonedas: Brave introduce un sistema de recompensas basado en el token BAT (Basic Attention Token), permitiendo a los usuarios ganar tokens al ver anuncios opcionales y apoyar a los creadores de contenido.

17. Generadores de documentación HTML (PHPDoc): PHPDocumentor, ApiGen, …

Nombre	Caracteristicas	Defectos	Etiquetas
PHPDoc	-Súper fácil de instalar y usar. -Crea una documentación excelente. -Aprende todo sobre tu código y lo utiliza para asegurarse de que tengas la mejor documentación. -Te proporciona un diagrama de clases UML para tu proyecto utilizando PlantUML o GraphViz. -Admite las últimas funciones que ofrece PHP. -Utiliza la información de tus DocBlocks para proporcionar aún más detalles. -Búsqueda de texto completo -Intégralo en tu canalización de CI con nuestra acción de Github o imagen de Docker.		Enlace